# Secure AI Audit Assistant with RAG and Graph-based RBAC

Advisor : Andrew Bond

Team : Apoorva Shastry
Ananya Praveen Shetty
Junie Mariam Varghese
Rinku Tekchandani

# The Audit Problem

Modern compliance teams face a growing gap between the volume of regulatory requirements and the tools available to meet them.

### Fragmented Sources

Auditors spend significant time searching across disconnected compliance documents with no unified query layer.

### Contextual Blindness

Traditional keyword search lacks semantic understanding, missing relevant policy context and relationships.

### Post-Retrieval Risk

Access control applied after retrieval exposes sensitive snippets before authorization is verified.

### Regulatory Pressure

Organizations face accelerating compliance mandates with no scalable, auditable tooling to keep pace.

# Project Objectives

Five core goals define the system's design, each addressing a specific gap in current audit tooling.

01

## Secure RAG Implementation

Retrieval-Augmented Generation with access control enforced at every pipeline stage.

02

## Natural Language Queries

Auditors ask questions in plain English, not Boolean syntax.

03

## Graph-Based RBAC via Neo4j

Model user-to-document permissions as a traversable property graph.

04

## Explainable Responses

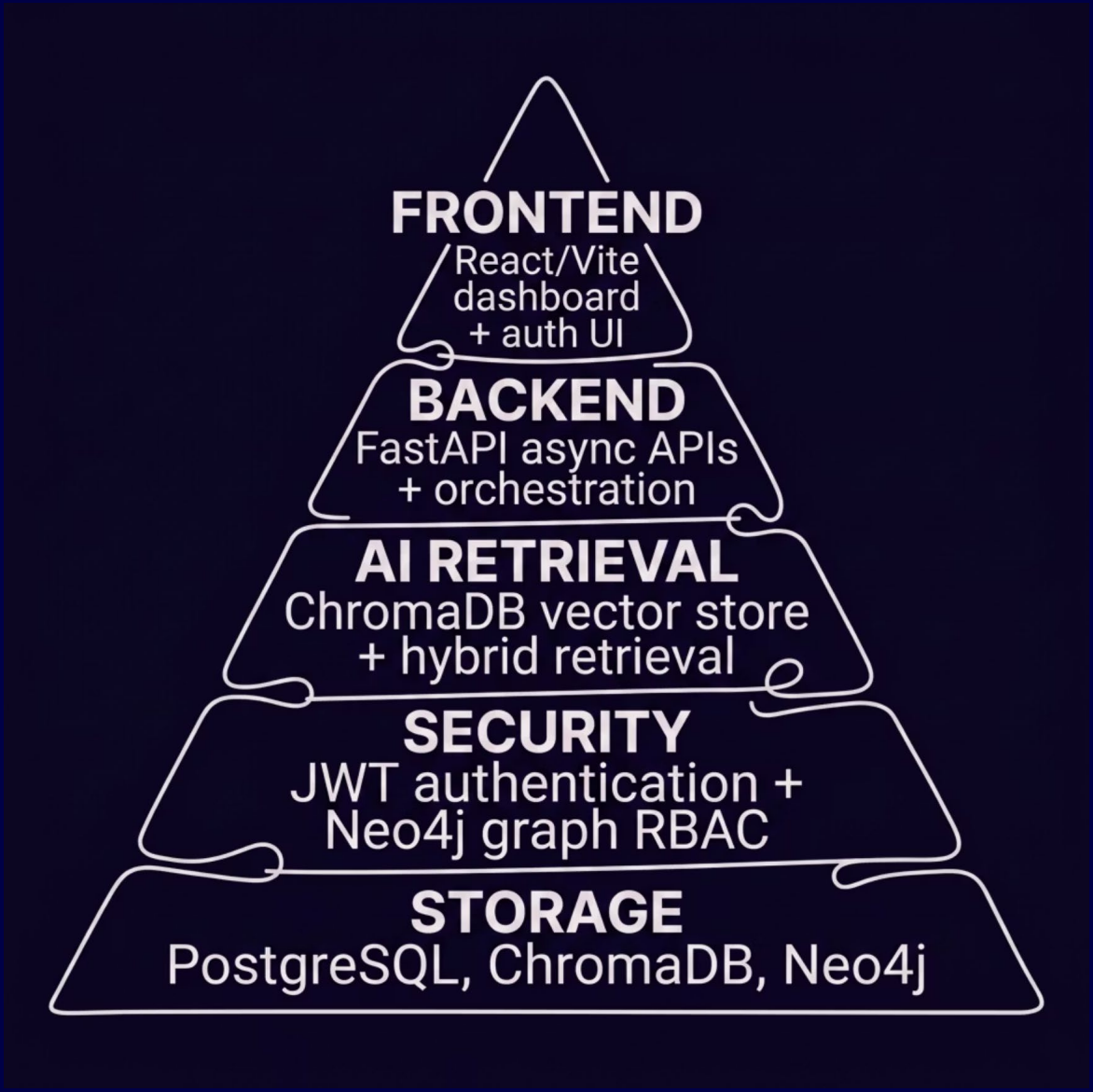
Every AI answer includes grounded citations traceable to source documents.

05

## Tamper-Evident Logging

Immutable audit logs with SHA-256 hash chaining for integrity verification.

# System Architecture



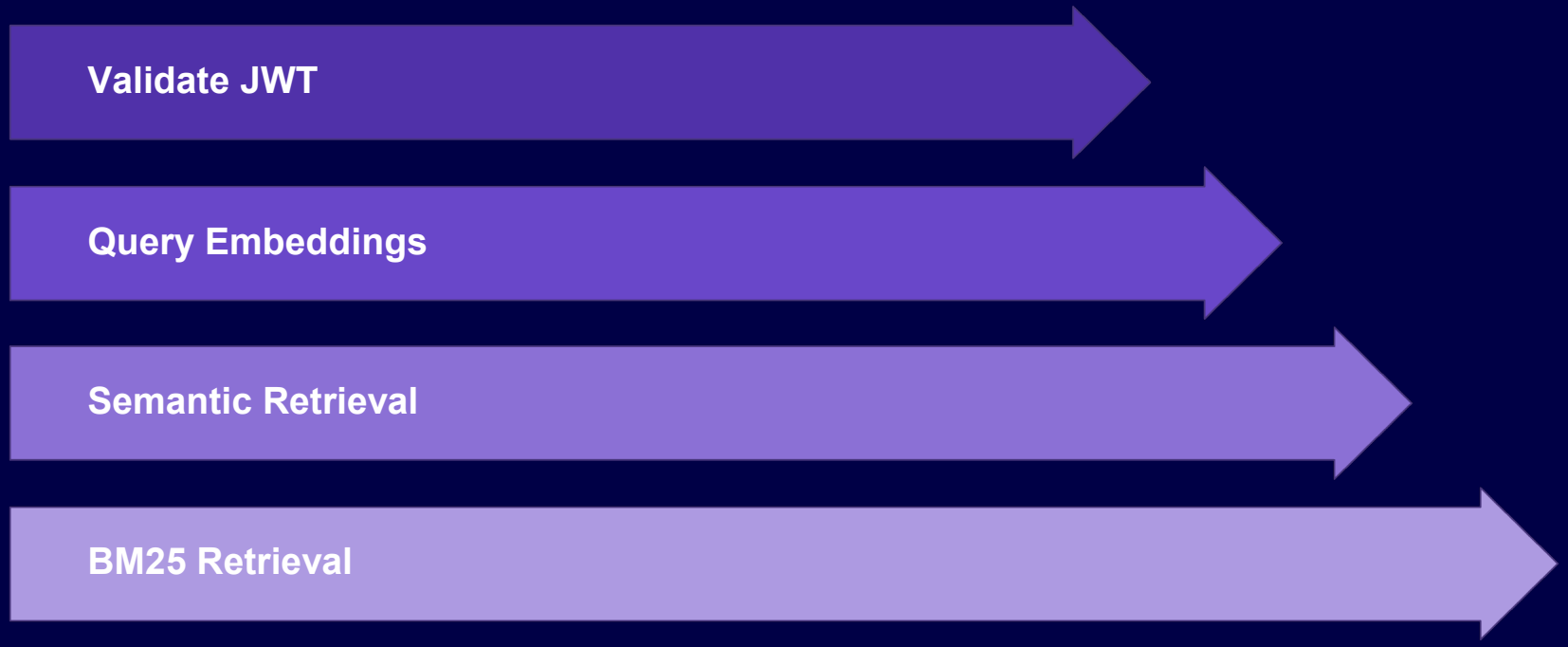

## Layered Security Design

Each architectural layer has a distinct responsibility, ensuring separation of concerns and defense-in-depth.

- **Frontend:** React + Vite dashboard with secure session management
- **Backend:** FastAPI async APIs handling orchestration and business logic
- **AI Retrieval:** ChromaDB vector store with hybrid semantic + keyword search
- **Security:** JWT authentication layered with Neo4j graph-based RBAC
- **Storage:** PostgreSQL, ChromaDB, and Neo4j each serving distinct data roles

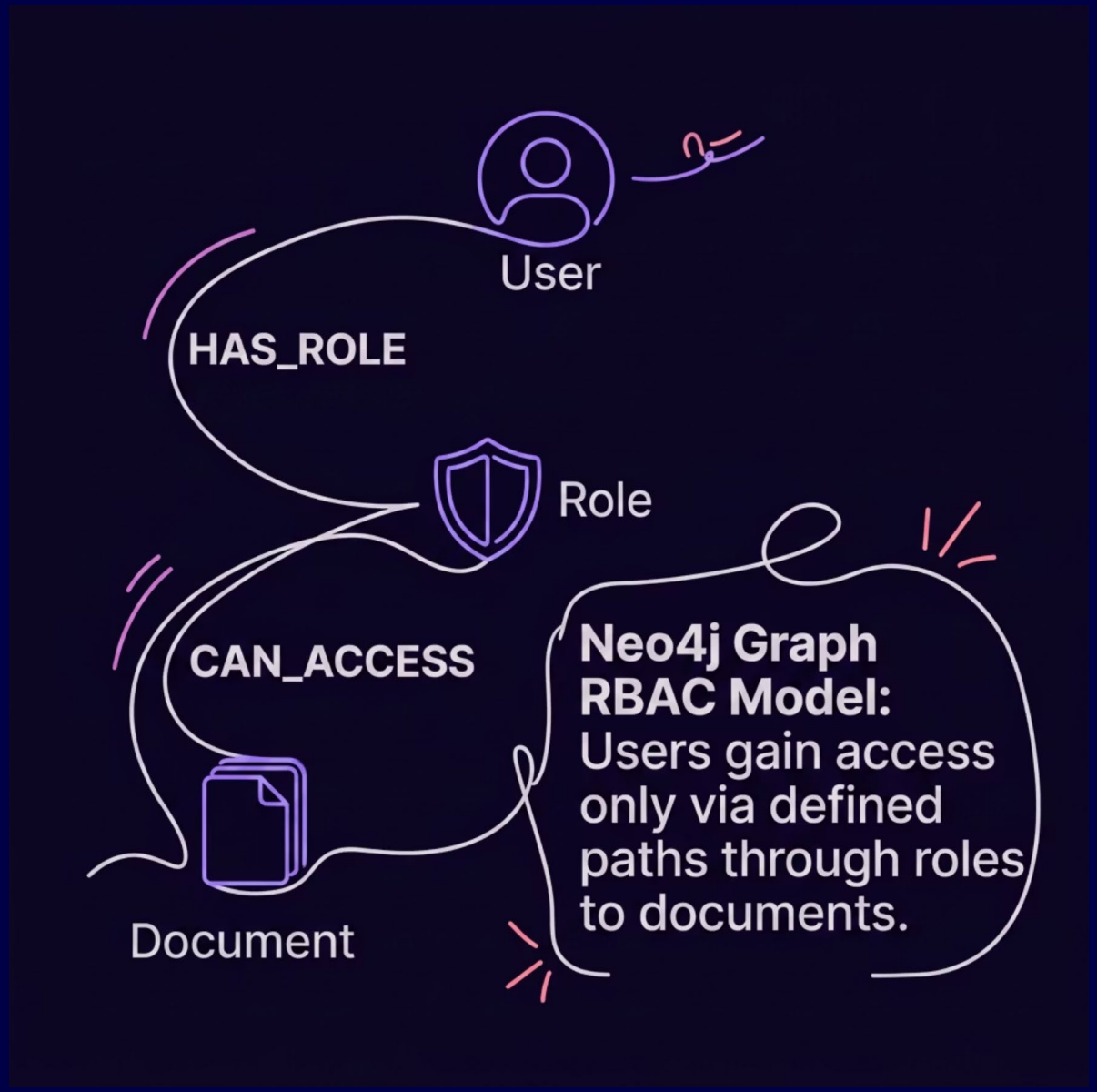
# RAG Pipeline: From Query to Citation

Every user query passes through a tightly controlled pipeline. Security checks occur **before** any LLM call — ensuring no unauthorized content is ever surfaced.




**Key security boundary:** RBAC filtering at step 6 ensures unauthorized document snippets never reach the LLM — a deny-by-default posture enforced programmatically.

# Graph-Based RBAC with Neo4j




### Access Control at the Graph Level

Permissions are modeled as traversable graph relationships — `User → Role → Document` — enabling fine-grained, auditable access decisions.

### Deny by Default

No implicit access. Every permission must be explicitly granted via graph relationship.

### Pre-Retrieval Enforcement

RBAC filters results before they reach the LLM — zero unauthorized snippet leakage.

### Opaque Denials

Unauthorized users receive: *"No results (insufficient access)"* — no information

# Tamper-Evident Audit Logging

Every security-relevant action is recorded in an immutable log protected by cryptographic hash chaining — providing verifiable evidence of system integrity.

## Comprehensive Coverage

All security-relevant actions logged: user, action type, timestamp, and outcome.

## SHA-256 Hash Chain

Each log entry includes a hash of the previous entry, creating a tamper-evident chain.

## Verify Chain Endpoint

A dedicated API endpoint validates the full hash chain, detecting any post-hoc modifications.

## Flexible Export

Audit logs support both CSV and JSON export for external compliance reporting and analysis.

# Testing & Validation

## Multi-Layer Testing Strategy

A comprehensive testing pyramid ensures correctness at every level — from isolated units to full end-to-end workflows under realistic conditions.


### Unit Tests

Pytest (backend) and Vitest (frontend) validate individual components in isolation.

### Integration Tests

Cross-system tests cover PostgreSQL, Neo4j, and ChromaDB interactions.

### E2E Tests

Playwright automates full user workflows through the React dashboard.

### Security Tests

Dedicated tests validate JWT integrity, RBAC enforcement, and audit log tamper-evidence.

**95%+ test success rate** achieved across all test layers, confirming system reliability under both normal and adversarial conditions.

# Performance & Results

The system delivers fast, secure retrieval at scale — with security overhead adding minimal latency to the overall response pipeline.

## ~1250ms

### Avg Response Time

End-to-end latency from query submission to grounded, cited answer.

## ~25ms

### RBAC Overhead

Graph-based access control adds negligible latency to each request.

## ~50ms

### ChromaDB Retrieval

Vector search performance at production document scale.

## 39.9%

### Memory Utilization

RBAC Mem utilization average.

# Conclusion & Future Work

The Secure AI Audit Assistant demonstrates that **security and AI capability are not in tension** — they reinforce each other when designed together from the ground up.

## What We Built

A RAG-powered audit assistant with graph-based RBAC, explainable citations, and tamper-evident logging — all enforced before any LLM call.

## What It Achieves

Faster audit workflows, zero unauthorized data leakage, and cryptographically verifiable audit trails for compliance reviewers.

## What's Next

Attribute-Based Access Control (ABAC), multimodal document ingestion, and enterprise SSO integrations for production deployment.